VoiceOps Founding AI Engineer Trial

Build us a customer-facing chatbot.

Our customers are sales managers and sales leaders, not technical people. They have a bunch of data sitting in a database from VoiceOps: their reps' calls, transcripts, coaching skill scores, insights extracted from conversations, competitive intelligence, all that. Right now they can't really see what's in it or get anything useful out of it. The chatbot is how we fix that. They talk to it the way they'd talk to anyone on their team. Stuff like "how is Sarah doing this month," "what are reps saying when customers bring up State Farm," "show me the calls where someone tried to save a renewal and lost it." The bot figures out the rest.

Make it conversational. Avoid the pattern where the bot takes a question, breaks it into steps, prints out a plan, and waits for permission. People don't talk like that.

We'll send you the dataset. It's a Postgres dump with synthetic data for a made-up org called Acme Insurance. About six reps and 200 calls, with transcripts, call metadata, skill classifications, and product-insight extractions. Enough shape to build something real on top of.

The agent runs in a sandbox so it can do whatever it needs in there. Write files, run queries, build dashboards, generate images, show all of that back to the user. A folder on your laptop is a totally fine sandbox. If you have time to wrap it in a container, that's next level.

The bot should be able to make images and show them inline. It should be able to make artifacts: dashboards the user can keep open, filter, drill into, come back to later, and ask the bot to tweak.

How it looks and feels matters a lot. The chat surface, the inline images, the dashboards. None of it should look like a generic engineering tool. The people using this are sales managers, and they care about the visuals. Take inspiration from ChatGPT, Claude, and the chatbots people already know. That's roughly the bar for the chat side. For the artifacts and dashboards, think about what a sales manager actually wants to look at, not what an engineer would build for themselves. Default matplotlib palettes won't cut it. Ideally there's a skill or system prompt baked in that pushes the agent toward customer-grade output by default, so you aren't fixing every chart by hand.

How you tackle this is up to you. We're not grading on a rubric.